

FINAL PROJECT PROPOSAL REVISED

SCAPEBUDDY

My solution for my personal MMO problems... and its alot of problems...

WHO IS THIS FOR?

The main target audience for this app is MMO players who are really forgetful and need to be reminded of useful info when playing the game, such as gear needed to be brought for a certain task, boss patterns, and updated meta for PVE or PVP content.

CORE FEATURES

#	Feature	Still in the MVP?	Flutter pieces it needs	Honest estimate
1	Login	keep	StatefulWidget,Scaffold,Columns,Texts ,TextFields,TextEditingController,obscureText,ElevatedButton,TextButton,Navigator,SnackBar	E.g. 10 hours
2	Boss Guides / Bestiary	keep	StatelessWidget,ListView.builder, ListTile,Inkwell,Navigator.push,MaterialPageRoute,Columns,Rows,Texts,Images,Containers,Cards	E.g. 10 hours
3	Build Dictionary	keep	StatefulWidget,GridView.builder, Inkwell,Container,showDialog(), AlertDialog,ListView.builder,ListTile,setState(),Images	E.g 10-20+ hours
4	Milestone Tracker	keep	StatefulWidget,Textfield,TextEditingController,Padding,InputDecoration, initState(),dispose()	E.g. 10 hours
5	Patchnotes Page	Cut	No Longer Included	N/A

STRETCH AND CUT GOALS

- **AI market helper**

Discarded this to be more realistic with my project

- **AI buddy for general game questions**

I personally look to talk to players in game for questions so I'll scrap this to focus more on what I need.

- **Patchnotes Page**

The game already had a QoL update allowing you to access patch notes while in the game so there's no need for this feature anymore

- **Login Screen (NEW)**

I added this one to be able to implement firebase so I can securely store the data of my users and so that they see their own unique builds and notes when using my app.

HOW MY APP SAVES DATA

Two different people should not see the same personal data. Each user's saved loadout and notes should be connected to their Firebase Authentication user ID, so users can only access their own data. The Bestiary is different because its monster information is shared and read-only for everyone.

For one user, I'd say probably around 10 records per week, depending on how often they change their loadout and save notes. The Bestiary does not create user records because its information is already stored in the app. If you save one loadout as one Firestore document and one note as one document, the amount of actual stored data can be small or smaller.

For my choice of storing data I am planning to use Firebase Cloud Firestore with Firebase authentication, I've been recommended to use this for my project as I asked questions with upperclassmen on what

they've used and I've put it into consideration as it can greatly help in making my project possible.

I chose Firebase because it allows the app to store user data in the cloud while separating each user's notes and loadouts using their Firebase user ID. The tradeoff I accepted is that Firebase is more complicated than local storage and depends on an internet connection, but it gives me cloud storage, authentication, and easier synchronization across devices.

For what I save concretely it's gonna be the slots and loadouts and the other aspects will be the Notepad which has the user's notes. The bestiary would not need to save user data since it stores and displays information and not user inputs. Now, have I tried it yet? No, and everything will be new to me so I'm gonna have to spend a lot of time going back and forth with modules and scouring the internet.

ONE THING THAT I WANT TO ADD THAT THE COURSE DID NOT TEACH:

Package: `firebase_core`, `firebase_auth`, and `cloud_firestore`.

Platform: Firebase Authentication and Cloud Firestore support Android, iOS, and Web, so the features can run in the browser during development.

Web fallback: Not needed because Firebase works on Web. I can use the actual Firebase implementation in the browser and record the mobile version for the final demonstration.

Priority: Firebase is responsible for the login system and for saving the user's Build Creator loadouts and Notepad data. If Firebase is removed, the app would lose its user accounts and cloud-saved personal data, so I need to start implementing Firebase early rather than treating it as a stretch goal.

HOW MY PROJECT RUNS WHEN SOMEONE ELSE OPENS IT

I am keeping the device_preview wrapper: Yes

My app runs in a browser with flutter run -d web-server, start to finish, with every screen reachable: not yet, testing will be needed to answer this.

Anything that needs real hardware degrades to sample data instead of crashing: No, this project does not require any external hardware

My project will live in a public repository in my own GitHub account, not the course org. Does anything I plan to build need an API key, a password, or real personal data?

The project will use Firebase, so it will require Firebase configuration and API key information to connect the Flutter app to Firebase. Users will also need an email and password when creating an account through Firebase Authentication. However, the project does not require real personal data. Test accounts and sample information can be used during development. The user's notes and loadouts will be stored under their Firebase user account.

DATA THE APP NEEDS TO REMEMBER

Thing	Fields	Where it saved
User Account	Email, Password	Firebase Auth
Loadout	Slot 1-12, Gear selected	Firestore

Notepad	User ID, Note text, Date/Time updated	Firestore
Bestiary	Name, Type, Description, Skills, Guide, Image	Local app data
Firebase Config	Project ID, API key, App Config	Project Config
Passwords	User password	Firebase Auth

FINAL SCREENS

- 1. LOGIN SCREEN**
- 2. HOME SCREEN**
- 3. BESTIARY**
- 4. MILESTONE TRACKER / NOTEPAD / REMINDER**
- 5. BUILD SETUP / LOADOUT / CREATOR**

RISK REVISED

The risk I named last time: Since I did not state any actual risks in the prelims I have decided to now state the real risks in the next question.

A new risk I did not see before: implementing databases / firebase, it is my first time working with this so it will be risky whether or not i can actually implement it into my project. But I will do my best to try.

For each: what is my first step to reduce it, and when? A date, not "eventually": My first step is to start researching how it is used properly in other people's projects and how I can replicate their success in my project so I can implement firebase properly.

WHAT CHANGED AND WHY:

SECTION	PRELIM SAID	NOW SAYS	WHY IT CHANGED
Core features	Prelim included a screen/page where you can view game patch notes	It is no longer included and has been cut from the core features.	As stated before; that the game already already had a QoL update allowing you to access patch notes while in the game so there's no need for this feature anymore.
Core features	No log in screen was included in the original plan	A log in is now a part of the project and core features.	This is so that users can authenticate themselves and allow them to store the notes and loadouts inside the app through Firebase.